SI100B: Introduction to Information Science and Technology

Lecture 7

By Ana Bell of MIT

LISTS

LISTS

- **Indexable ordered sequence** of objects
  - Usually homogeneous (i.e., all integers, all strings, all lists)
  - But can contain mixed types (not common)
- Denoted by **square brackets**, [ ]
- **Mutable**, this means you can change values of specific elements of list

Notes: Tuples were: ()

Remember **tuples** are **immutable** -- you cannot change element values.
Lists are **mutable**, you can change them directly.

INDICES and ORDERING

Remember strings and tuples?

```
empty_list = []
short_list = [2]
hello_list = [2, "a", 4, [1, 2]]
hello_list[0]  # 2
hello_list[3]  # [1, 2]
hello_list[4]  # IndexError: list index out of range
hello_list[1:2]  # = ["a"]
hello_list[1:3]  # = ["a",4]

[1, 2] + [3, 4]  # [1, 2, 3, 4]
[1, 2] * 2  # [1, 2, 1, 2]

len(hello_list)  # 4
max([3, 5, 0])  # 5
hello_list[0] = 3
```

← Empty list

✓ Mutable (supports modification)

ITERATING OVER a LIST

- Like string or tuple, can iterate over elements of list directly

```
L = [3, 5, 0]
total = 0
for v in L:
    total += v
```

YOU TRY IT!

- Write a function that meets these specs:

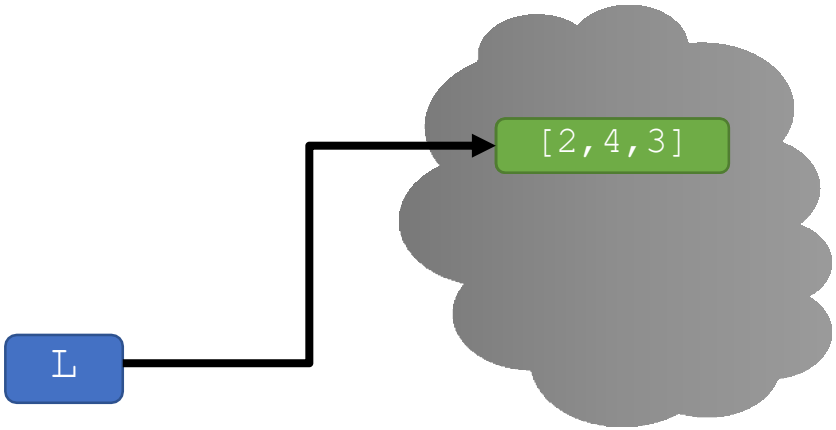
```
def sum_and_prod(L):
    """
    L is a list of numbers
    Return a tuple where the first value is the
    sum of all elements in L and the second value
    is the product of all elements in L
    """
    # ... your code ...
```

MUTABILITY

MUTABILITY

- Lists are **mutable**!
- Assigning to an element at an index **changes** the value

```
L = [2, 4, 3]
L[1] = 5
```

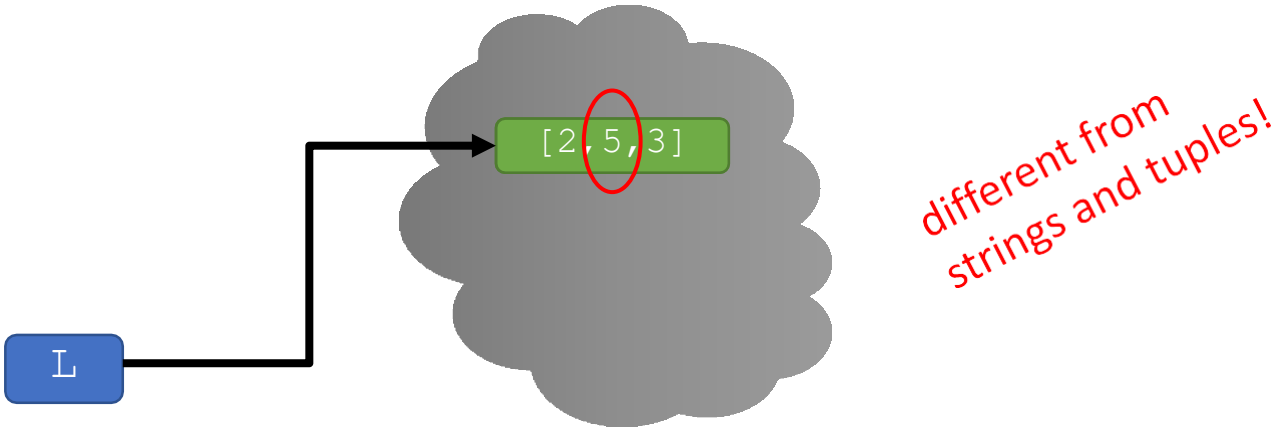


MUTABILITY

- Lists are **mutable**!
- Assigning to an element at an index **changes** the value

```
L = [2, 4, 3]
L[1] = 5
```

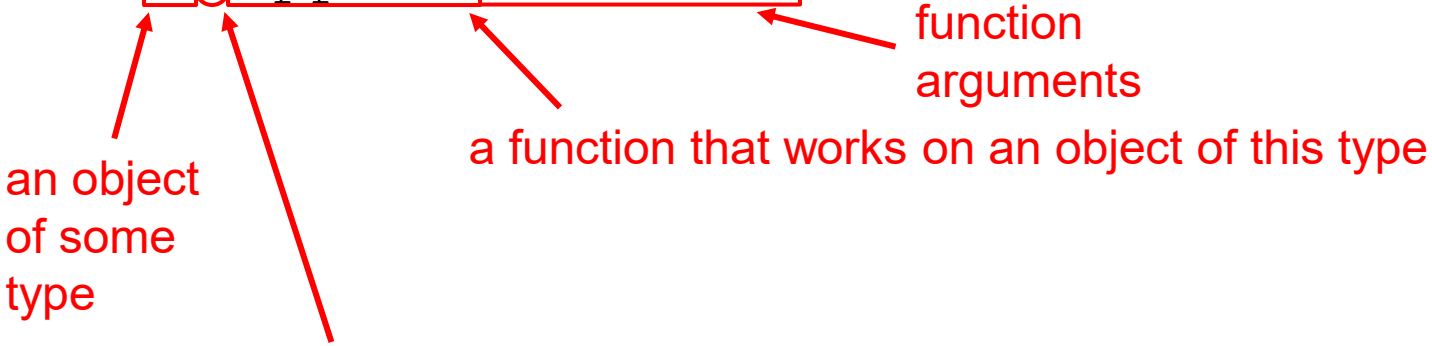
- `L` is now `[2, 5, 3];` note this is the **same object** `L`



OPERATION ON LISTS: append

- **Add** an element to end of list

```
L.append(element)
```



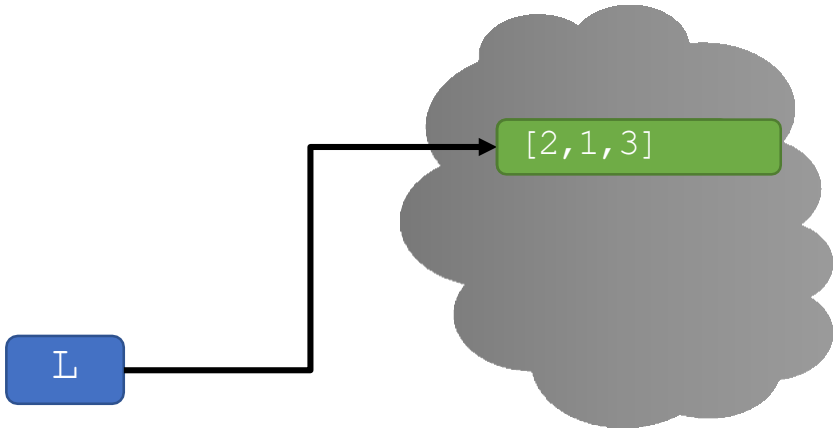
- What is the dot?
  - Lists are Python objects, everything in Python is an object
  - Object types have **associated operations**
  - Access them by `object_name.do_something()`
  - Can be seen as calling `do_something` with argument `object_name`

OPERATION ON LISTS – append

- **Add** an element to end of list with `L.append(element)`
- **Mutates** the list!

```
L = [2,1,3]
L.append(5)
```

→ L is now `[2,1,3,5]`

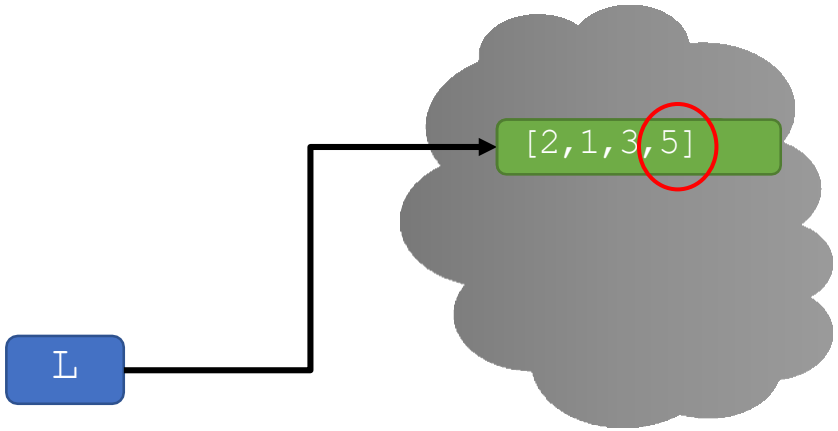


OPERATION ON LISTS – append

- **Add** an element to end of list with `L.append(element)`
- **Mutates** the list!

```
L = [2,1,3]
L.append(5)
```
→ L is now `[2,1,3,5]`

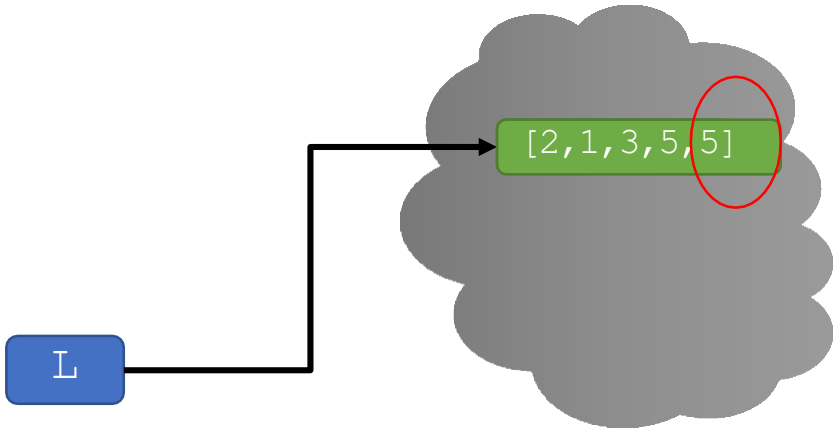
```
L = L.append(5)
```



OPERATION ON LISTS – append

- **Add** an element to end of list with `L.append(element)`
- **Mutates** the list!

```
L = [2,1,3]
L.append(5)  → L is now [2,1,3,5]
L = L.append(5)
```

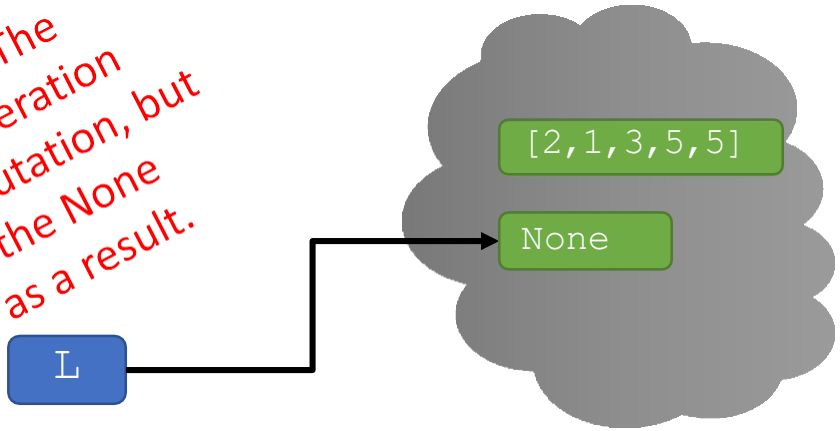


OPERATION ON LISTS – append

- **Add** an element to end of list with `L.append(element)`
- **Mutates** the list!

```
L = [2,1,3]
L.append(5)  → L is now [2,1,3,5]
L = L.append(5)
```

Be careful! The append operation does a mutation, but returns the None object as a result.



Append is used strictly for its **side effect**

BIG IDEA

Some functions mutate the input and don't return anything.

We use these functions for their side effect.

YOU TRY IT!

- What is the value of L1, L2, L3 and L at the end?

```
L1 = ['re']
L2 = ['mi']
L3 = ['do']
L4 = L1 + L2
L3.append(L4)
L = L1.append(L3)
```

YOU TRY IT!

- Write a function that meets these specs:

```
def make_ordered_list(n):
    """
    n is a positive int
    Returns a list containing all ints in order
    from 0 to n (inclusive)
    """
```

YOU TRY IT!

- Write a function that meets these specs:

```
def remove_elem(L, e):
    """
    L is a list
    Returns a new list with elements in the same order as L
    but without any elements equal to e.
    """

L = [1,2,2,2]
print(remove_elem(L, 2)) # prints [1]
```

STRINGS to LISTS

- Convert **string to list** with `list(s)`
  - Every character from s is an element in a list
- Use `s.split()`, to **split a string on a character** parameter, splits on spaces if called without a parameter

```
s = "I<3 cs &u?"
```
→ s is a string

```
L = list(s)
```
→ L is `['I','<','3',' ','c','s',' ','&','u','?']`

```
L1 = s.split(' ')
```
→ L1 is `['I<3','cs','&u?']`

```
L2 = s.split('<')
```
→ L2 is `['I', '3 cs &u?']`

LISTS to STRINGS

- Convert a **list of strings back to string**
- Use `''.join(L)` to turn a **list of strings into a bigger string**
- Can give a character in quotes to add char between every element

```
L =['a','b','c']
A = ''.join(L)
B = '_'.join(L)
C=''.join([1,2,3])
C=''.join(['1','2','3']
```

→ L is a list
→ A is "abc"
→ B is "a_b_c"
→ an error
→ C is "123" a string!

YOU TRY IT!

- Write a function that meets these specs:

```
def count_words(sen):
    """ sen is a string representing a sentence
    Returns how many words are in s (a word is a
    sequence of characters between spaces.
    """

print(count_words("Hello it's me"))
```

A FEW INTERESTING LIST OPERATIONS

- `sort()`

```
L = [4,2,7]
L.sort()
```

  - **Mutates** L

- `reverse()`

```
L = [4,2,7]
L.reverse()
```

  - **Mutates** L

- `sorted()`

```
L = [4,2,7]
L_new = sorted(L)
```

  - Returns a sorted version of L (**no mutation**!)

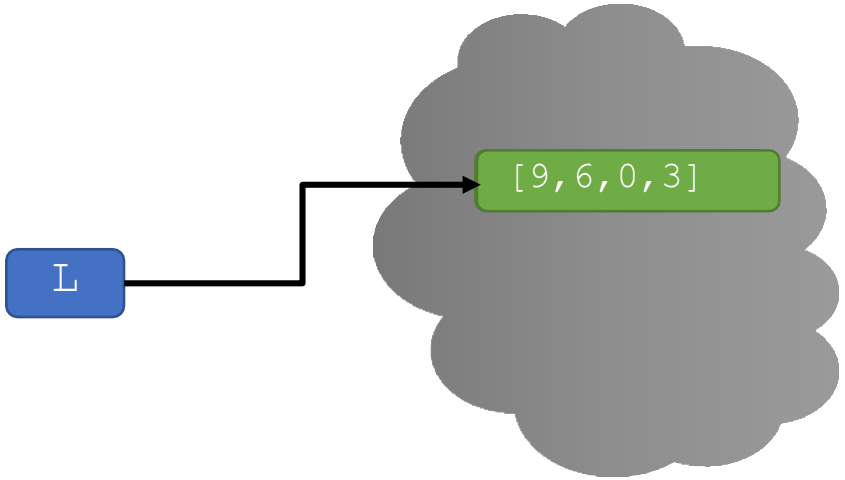
MUTABILITY

```
L=[9,6,0,3]
L.append(5)
```

`a = sorted(L)` → returns a **new** sorted list, does **not mutate** `L`

`b = L.sort()` → **mutates** `L` to be `[0,3,5,6,9]` and returns None

`L.reverse()` → **mutates** `L` to be `[9,6,5,3,0]` and returns None



MUTABILITY

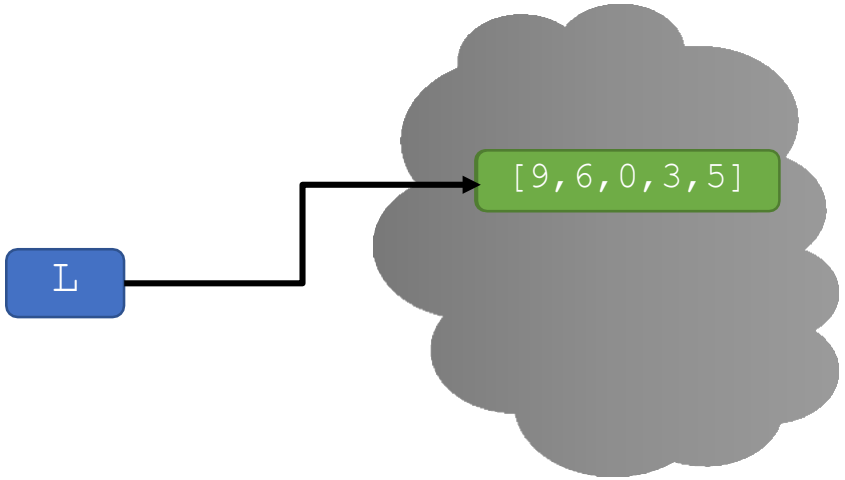
```
L=[9,6,0,3]
L.append(5)
```

append, sort, reverse all used for **side effect**

`a = sorted(L)` → returns a **new** sorted list, does **not mutate** `L`

`b = L.sort()` → **mutates** `L` to be `[0,3,5,6,9]` and returns None

`L.reverse()` → **mutates** `L` to be `[9,6,5,3,0]` and returns None



MUTABILITY

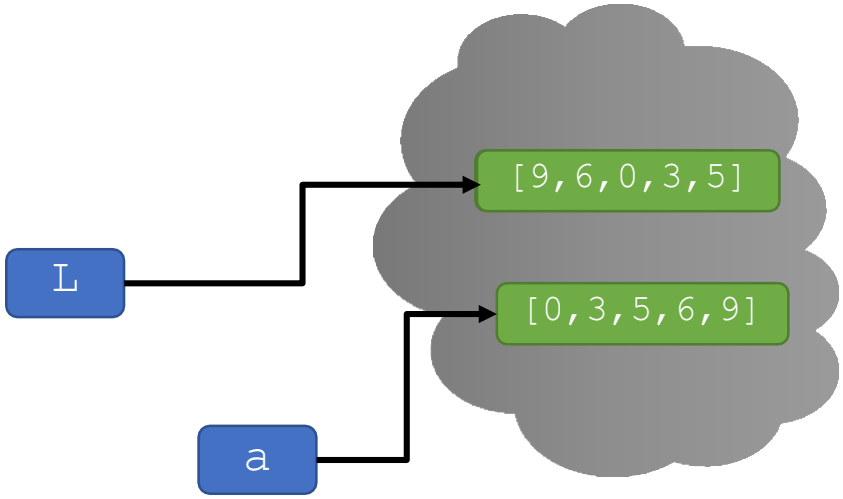
```
L=[9,6,0,3]
L.append(5)
```

append, sort, reverse all used for **side effect**

`a = sorted(L)` → returns a **new** sorted list, does **not mutate** `L`

`b = L.sort()` → **mutates** `L` to be `[0,3,5,6,9]` and returns None

`L.reverse()` → **mutates** `L` to be `[9,6,5,3,0]` and returns None



MUTABILITY

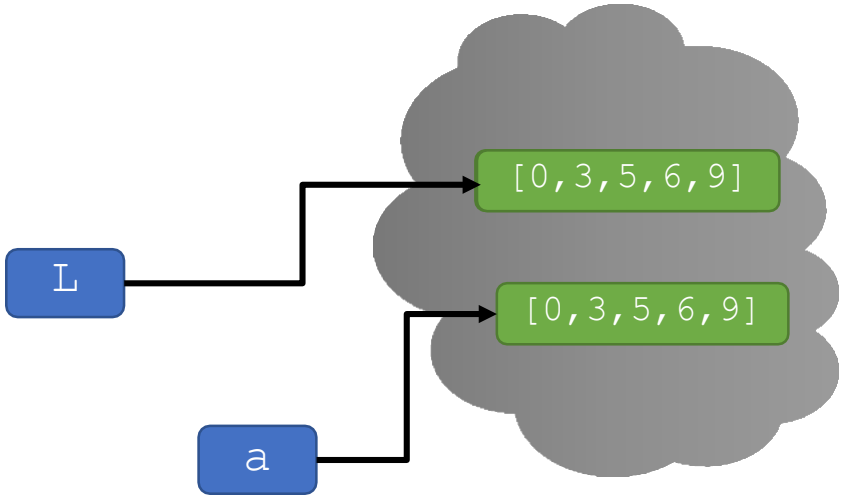
```
L=[9,6,0,3]
L.append(5)
```

append, sort, reverse all used for **side effect**

`a = sorted(L)` → returns a **new** sorted list, does **not mutate** `L`

`b = L.sort()` → **mutates** `L` to be `[0,3,5,6,9]` and returns None

`L.reverse()` → **mutates** `L` to be `[9,6,5,3,0]` and returns None



MUTABILITY

```
L=[9,6,0,3]
L.append(5)
```

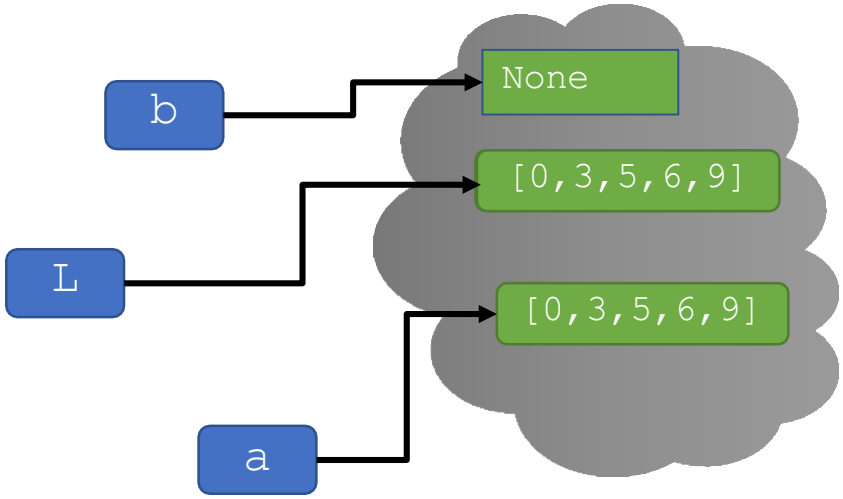
append, sort, reverse all used for **side effect**

`a = sorted(L)` → returns a **new** sorted list, does **not mutate** `L`

`b = L.sort()` → **mutates** `L` to be `[0,3,5,6,9]` and returns None

`L.reverse()` → **mutates** `L` to be `[9,6,5,3,0]` and returns None

Never do this.
Just use L.sort



MUTABILITY

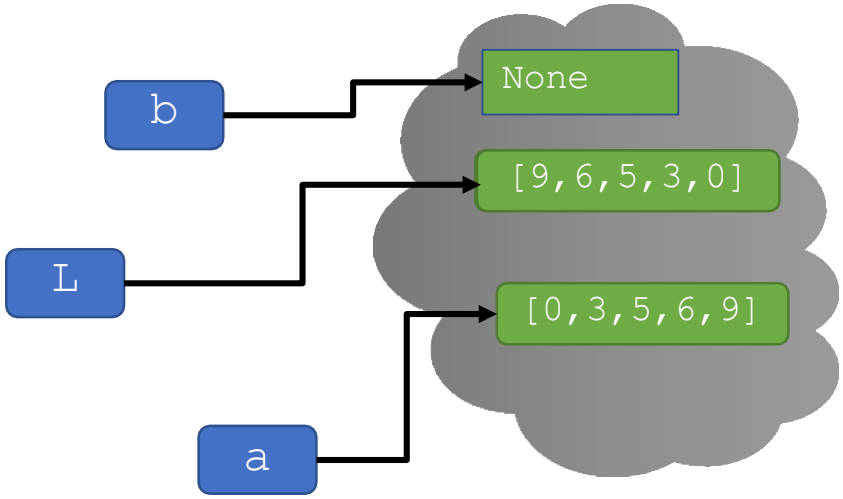
```
L=[9,6,0,3]
L.append(5)
```

append, sort, reverse all used for **side effect**

`a = sorted(L)` → returns a **new** sorted list, does **not mutate** `L`

`b = L.sort()` → **mutates** `L` to be `[0,3,5,6,9]` and returns None

`L.reverse()` → **mutates** `L` to be `[9,6,5,3,0]` and returns None



MUTABILITY

```
L=[9,6,0,3]
L.append(5)
```

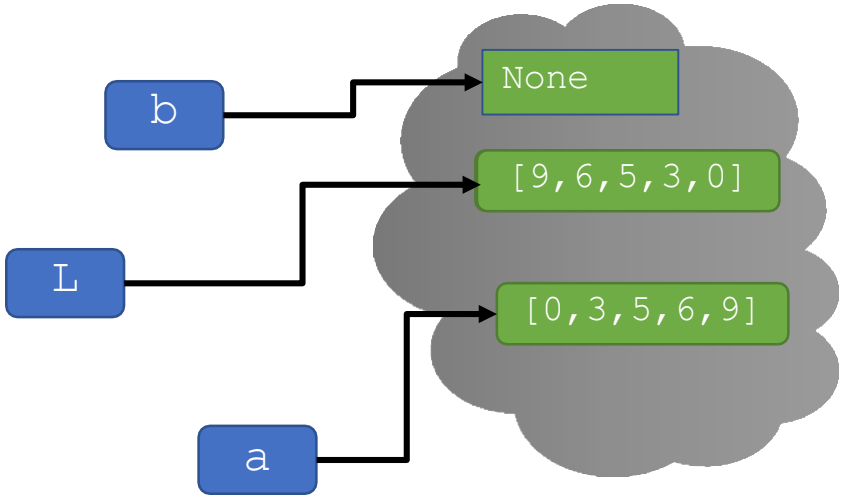
append, sort, reverse all used for **side effect**

`a = sorted(L)` → returns a **new** sorted list, does **not mutate** `L`

`b = L.sort()` → **mutates** `L` to be `[0,3,5,6,9]` and returns None

`L.reverse()` → **mutates** `L` to be `[9,6,5,3,0]` and returns None

Remember, we have to invoke the function even if it takes no arguments



YOU TRY IT!

- Write a function that meets these specs:

```
def sort_words(sen):
    """ sen is a string representing a sentence
    Returns a list containing all the words in sen but
    sorted in alphabetical order.
    """

print(sort_words("look at this photograph"))
```

LISTS SUPPORT ITERATION

- Let's write a **function that mutates the input**
- Example: square every element of a list, mutating original list

```
def square_list(L):
    for elem in L:
        # ?? How to do L[index] = the square ??
        # ?? elem is an element in L, not the index :(
```

- Solutions (we'll go over option 2, try the others on your own!):
  - Option 1: Make a **new variable** representing the index, initialized to 0 before the loop and incremented by 1 in the loop.
  - Option 2: **Loop over the index** not the element, and use L[index] to get the element
  - Option 3: Use **enumerate** in the for loop (I leave this option to you to look up). i.e. `for i,e in enumerate(L)`

LISTS SUPPORT ITERATION

- Example: square every element of a list, mutating original list

```
def square_list(L):
    for i in range(len(L)):
        L[i] = L[i]**2
```

To change elements of list, need to loop over indices into list

An assignment statement. L[i] is not a name, but points to a particular spot in the list data structure.

L[i] is the element

- Note, no return!

TRACE the CODE with an EXAMPLE

- Example: square every element of a list, mutating original list

```
def square_list(L):
    for i in range(len(L)):
        L[i] = L[i]**2

Lin = [2,3,4]
print("before fcn call:",Lin) # prints [2,3,4]
square_list(Lin)
print("after fcn call:",Lin) # prints [4,9,16]
```

The function mutates the input object passed in (Lin)

No variable to assign function call to!

BIG IDEA

Functions that mutate the input likely…..

Iterate over len(L) not L.

Return None, so the function call does not need to be saved.

MUTATION

- Lists are **mutable** structures
- There are many advantages to being able to **change a portion** of a list
  - Suppose I have a very long list (e.g. of personnel records) and I want to update one element. Without mutation, I would have to copy the entire list, with a new version of that record in the right spot. A mutable structure lets me change just that element
- But, this ability can also introduce unexpected challenges

TRICKY EXAMPLE 1: append

- **Range returns something that behaves like a tuple**
  (but isn't – it returns an *iterable*)

```
range(4)
range(2,9,2)
```

→ *kind of like* tuple (0,1,2,3)

→ *kind of like* tuple (2,4,6,8)

```
L = [1,2,3,4]
for i in range(len(L)):
    L.append(i)
    print(L)
```

Iteration sequence is pre-determined at beginning of loop

1st time: L is [1, 2, 3, 4, 0]

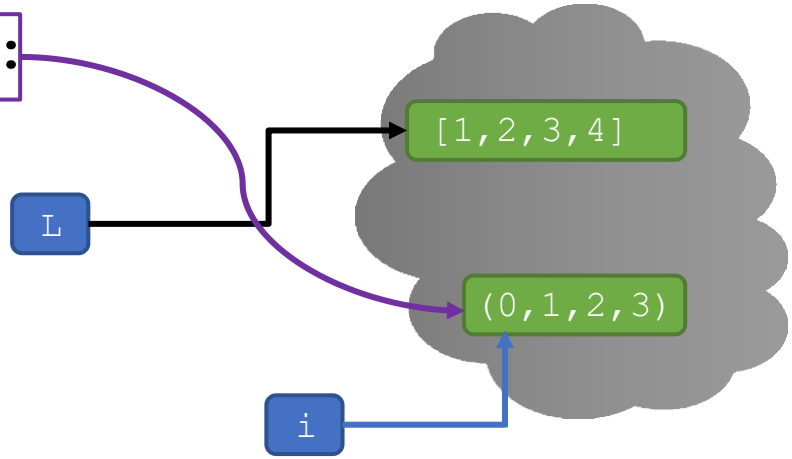
2nd time: L is [1, 2, 3, 4, 0, 1]

3rd time: L is [1, 2, 3, 4, 0, 1, 2]

4th time: L is [1, 2, 3, 4, 0, 1, 2, 3]

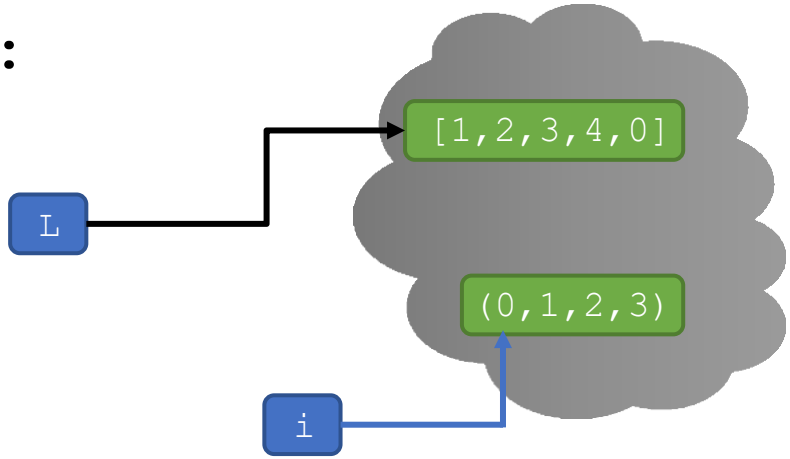
TRICKY EXAMPLE 1: append

```
L = [1,2,3,4]
for i in range(len(L)):
  L.append(i)
  print(L)
```



TRICKY EXAMPLE 1: append

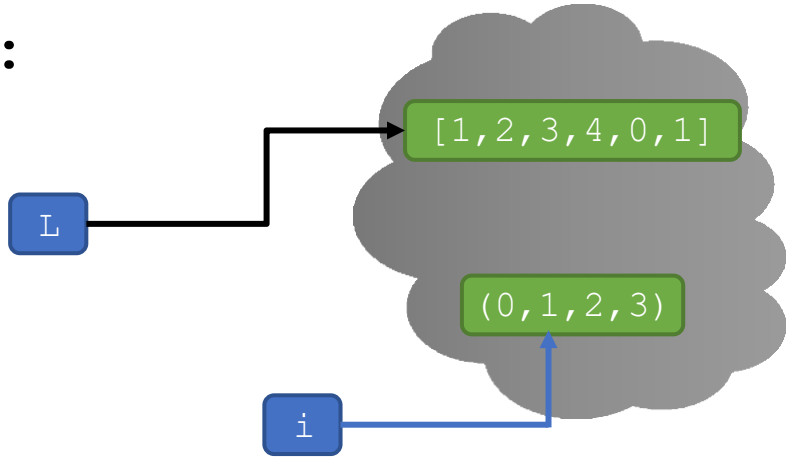
```
L = [1,2,3,4]
for i in range(len(L)):
  L.append(i)
  print(L)
```



1st time: L is [1, 2, 3, 4, 0]

TRICKY EXAMPLE 1: append

```
L = [1,2,3,4]
for i in range(len(L)):
  L.append(i)
  print(L)
```

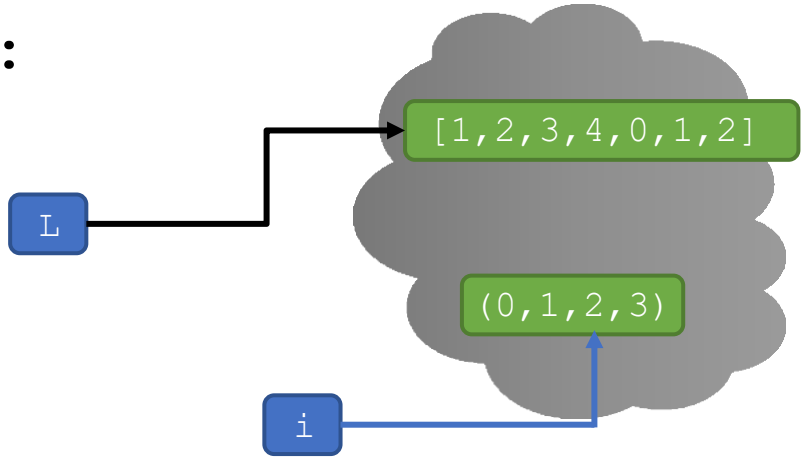


1st time: L is [1, 2, 3, 4, 0]

2nd time: L is [1, 2, 3, 4, 0, 1]

TRICKY EXAMPLE 1: append

```
L = [1,2,3,4]
for i in range(len(L)):
  L.append(i)
  print(L)
```



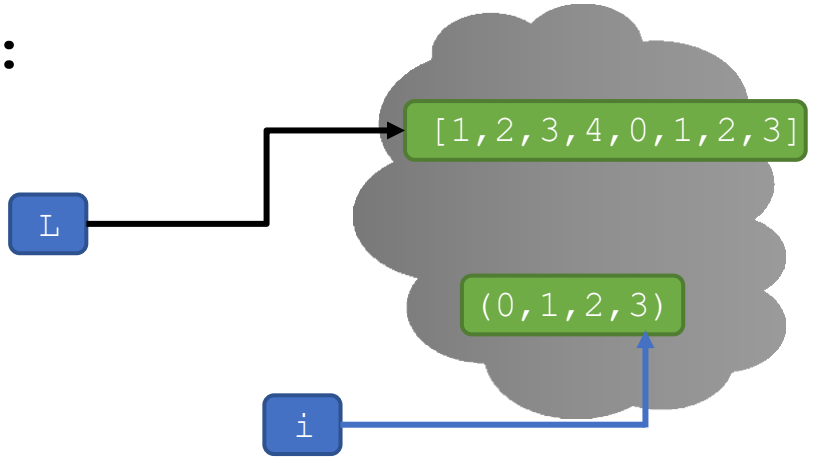
1st time: L is [1, 2, 3, 4, 0]

2nd time: L is [1, 2, 3, 4, 0, 1]

3rd time: L is [1, 2, 3, 4, 0, 1, 2]

TRICKY EXAMPLE 1: append

```
L = [1,2,3,4]
for i in range(len(L)):
  L.append(i)
  print(L)
```



End of iteration

1st time: L is [1, 2, 3, 4, 0]

2nd time: L is [1, 2, 3, 4, 0, 1]

3rd time: L is [1, 2, 3, 4, 0, 1, 2]

4th time: L is [1, 2, 3, 4, 0, 1, 2, 3]

`i` iterates over a "tuple" created by range; mutation of `L` does not affect this tuple

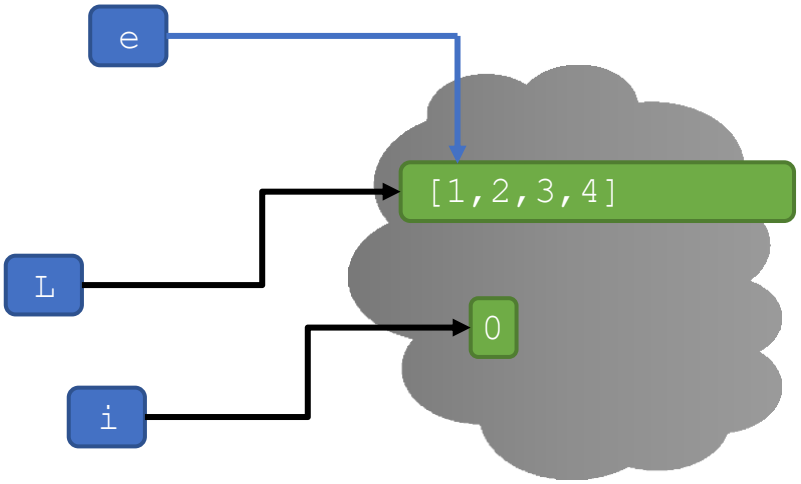
TRICKY EXAMPLE 2: append

- Looks similar but …

```
L = [1,2,3,4]
i = 0
for e in L:
    L.append(i)
    i += 1
    print(L)
```

Originally [1,2,3,4]

L is **mutated** each iteration



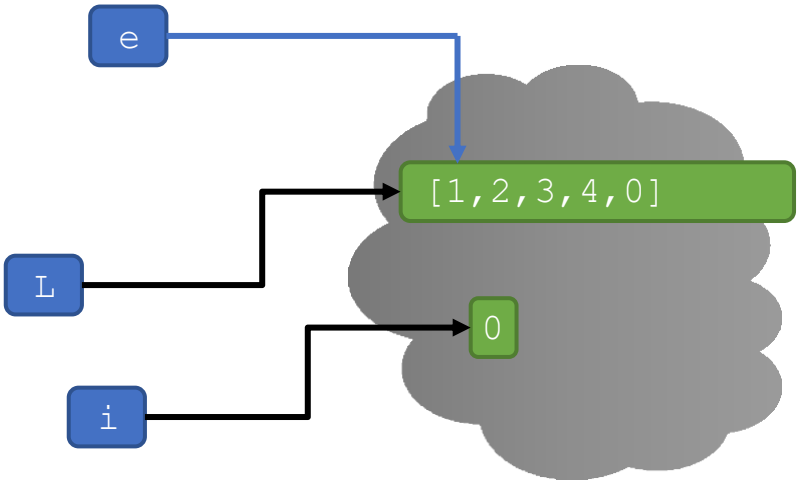
TRICKY EXAMPLE 2: append

- Looks similar but …

```
L = [1,2,3,4]
i = 0
for e in L:
    L.append(i)
    i += 1
    print(L)
```

Originally [1,2,3,4]

L is **mutated** each iteration



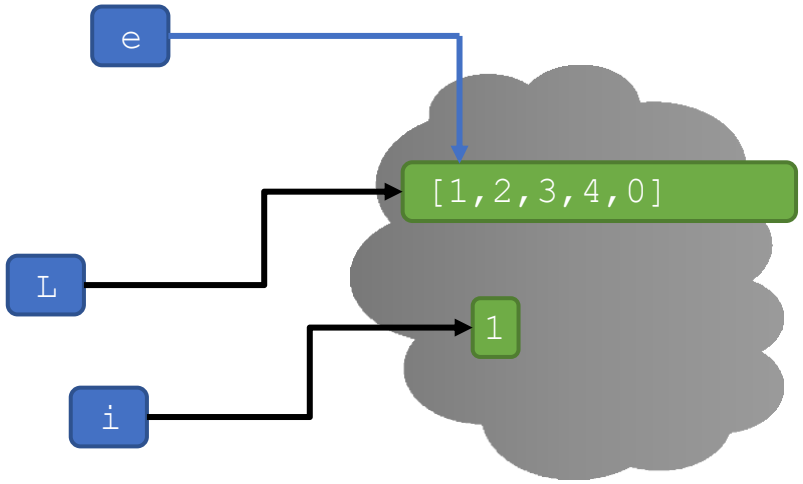
TRICKY EXAMPLE 2: append

- Looks similar but …

```
L = [1,2,3,4]
i = 0
for e in L:
    L.append(i)
    i += 1
    print(L)
```

Originally [1,2,3,4]

L is **mutated** each iteration



1st time: L is [1, 2, 3, 4, 0]

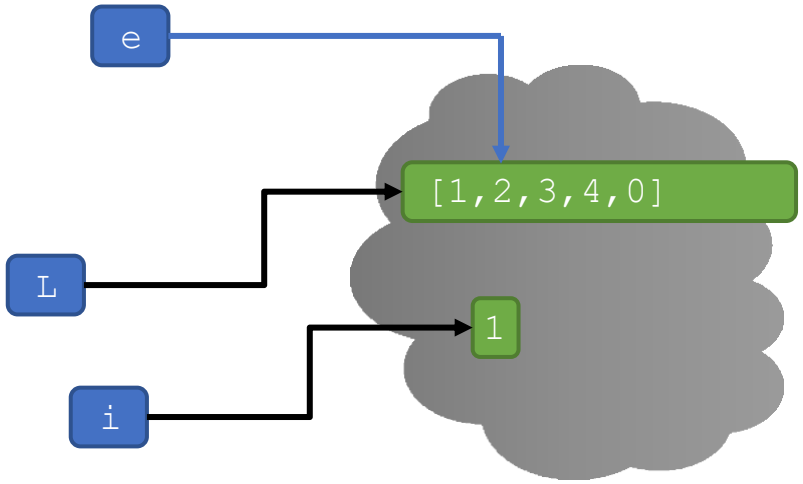
TRICKY EXAMPLE 2: append

- Looks similar but …

```
L = [1,2,3,4]
i = 0
for e in L:
    L.append(i)
    i += 1
    print(L)
```

Originally [1,2,3,4]

L is **mutated** each iteration



1st time: L is [1, 2, 3, 4, 0]

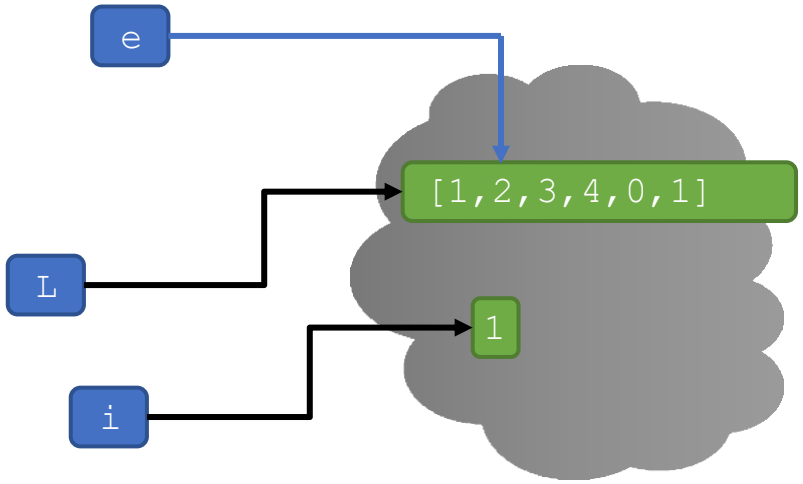
TRICKY EXAMPLE 2: append

- Looks similar but …

```
L = [1,2,3,4]
i = 0
for e in L:
    L.append(i)
    i += 1
    print(L)
```

Originally [1,2,3,4]

L is **mutated** each iteration



1st time: L is [1, 2, 3, 4, 0]

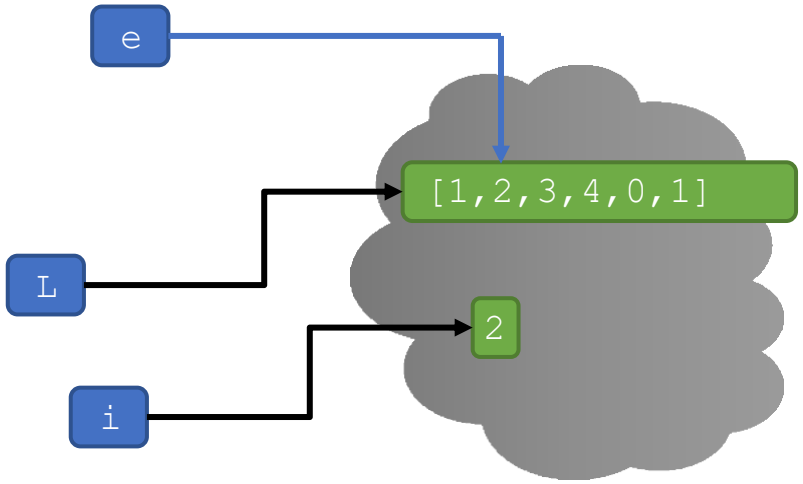
TRICKY EXAMPLE 2: append

- Looks similar but …

```
L = [1,2,3,4]
i = 0
for e in L:
    L.append(i)
    i += 1
    print(L)
```

Originally [1,2,3,4]

L is **mutated** each iteration



1st time: L is [1, 2, 3, 4, 0]

2nd time: L is [1, 2, 3, 4, 0, 1]

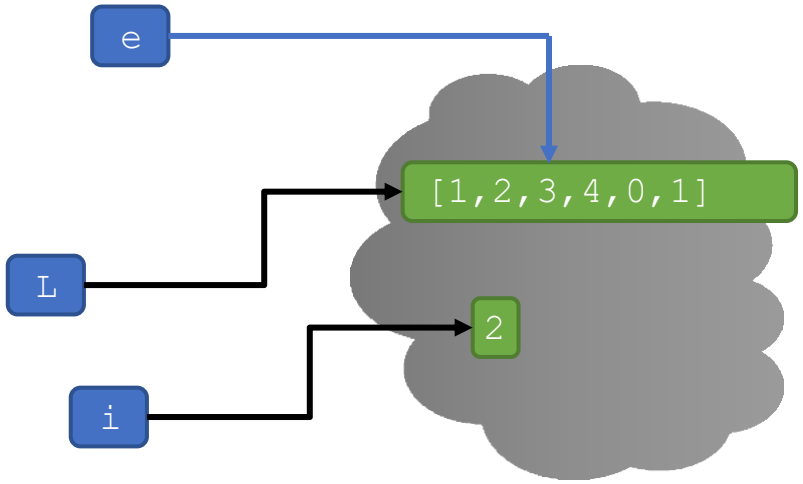
TRICKY EXAMPLE 2: append

- Looks similar but …

```
L = [1,2,3,4]
i = 0
for e in L:
    L.append(i)
    i += 1
    print(L)
```

Originally [1,2,3,4]

L is **mutated** each iteration



1st time: L is [1, 2, 3, 4, 0]

2nd time: L is [1, 2, 3, 4, 0, 1]

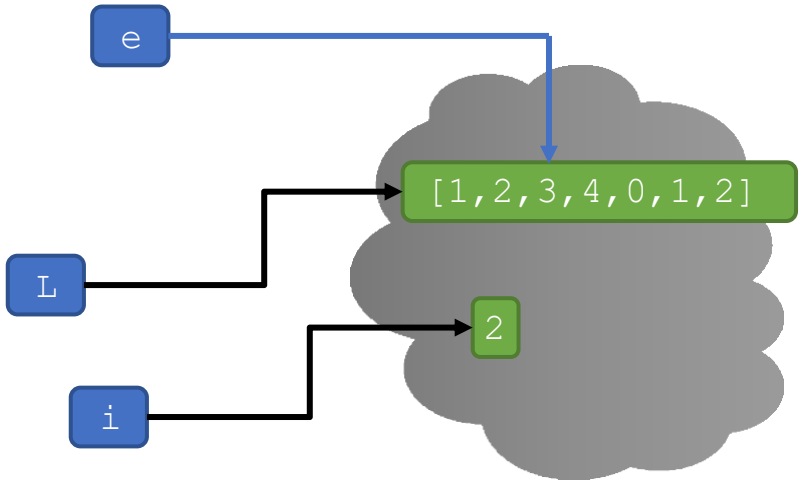
TRICKY EXAMPLE 2: append

- Looks similar but …

```
L = [1,2,3,4]
i = 0
for e in L:
    L.append(i)
    i += 1
    print(L)
```

Originally [1,2,3,4]

L is **mutated** each iteration



1st time: L is [1, 2, 3, 4, 0]

2nd time: L is [1, 2, 3, 4, 0, 1]

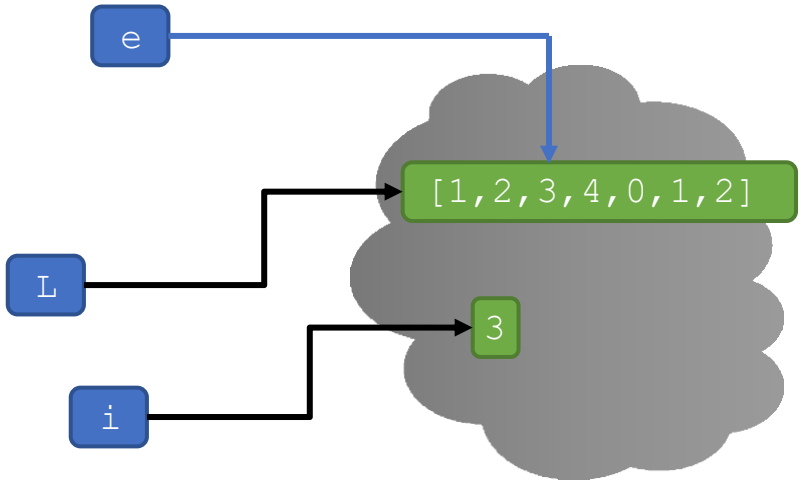
TRICKY EXAMPLE 2: append

- Looks similar but …

```
L = [1,2,3,4]
i = 0
for e in L:
    L.append(i)
    i += 1
    print(L)
```

Originally [1,2,3,4]

L is **mutated** each iteration



1st time: L is [1, 2, 3, 4, 0]

2nd time: L is [1, 2, 3, 4, 0, 1]

3rd time: L is [1, 2, 3, 4, 0, 1, 2]

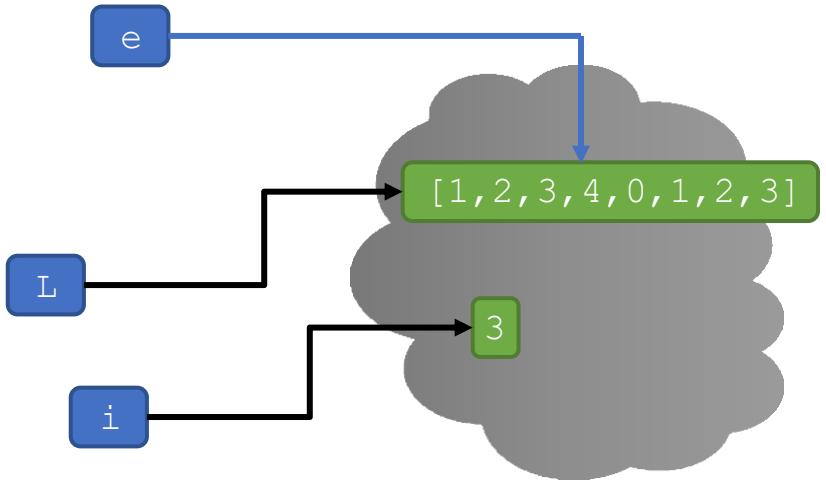
TRICKY EXAMPLE 2: append

- Looks similar but …

```
L = [1,2,3,4]
i = 0
for e in L:
    L.append(i)
    i += 1
    print(L)
```

Originally [1,2,3,4]

L is **mutated** each iteration



1st time: L is [1, 2, 3, 4, 0]

2nd time: L is [1, 2, 3, 4, 0, 1]

3rd time: L is [1, 2, 3, 4, 0, 1, 2]

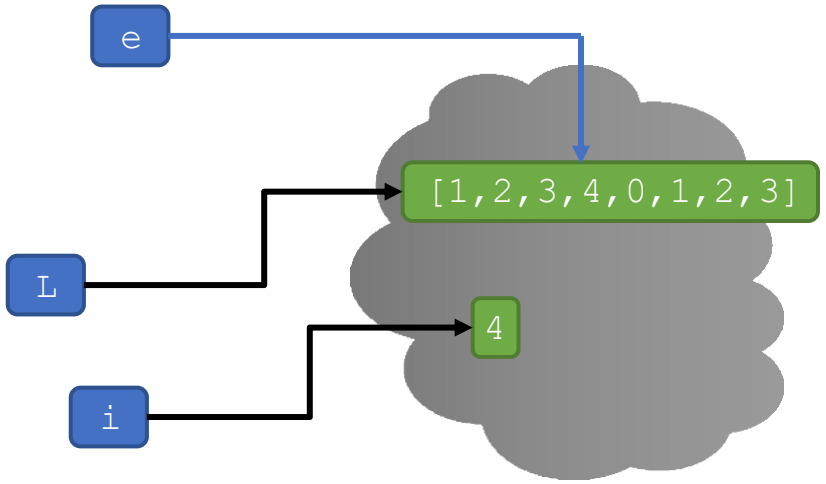
TRICKY EXAMPLE 2: append

- Looks similar but …

```
L = [1,2,3,4]
i = 0
for e in L:
    L.append(i)
    i += 1
print(L)
```

Originally [1,2,3,4]

L is **mutated** each iteration



In previous example, L was accessed at onset to create a range iterable; in this example, the loop is directly accessing indices into L

1st time: L is [1, 2, 3, 4, 0]

2nd time: L is [1, 2, 3, 4, 0, 1]

3rd time: L is [1, 2, 3, 4, 0, 1, 2]

4th time: L is [1, 2, 3, 4, 0, 1, 2, 3]

NEVER STOPS!

COMBINING LISTS

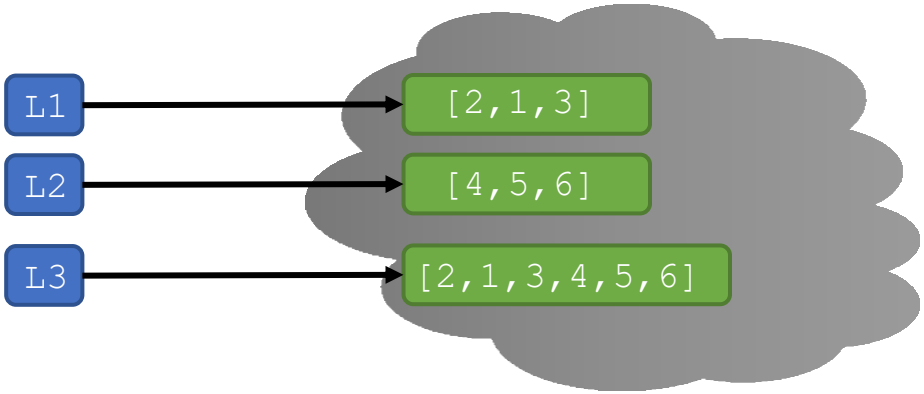
Remember strings

- **Concatenation**, + operator, creates a **new** list, with copies
- **Mutate** list with `L.extend(some_list)` (copy of `some_list`)

```
L1 = [2,1,3]
L2 = [4,5,6]
L3 = L1 + L2
L1.extend([0,6])
```

→ `L3` is `[2,1,3,4,5,6]`

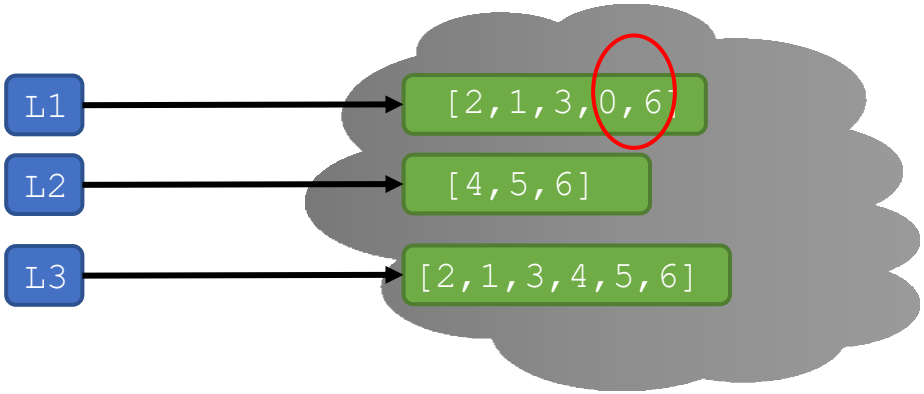
→ mutate `L1` to `[2,1,3,0,6]`



COMBINING LISTS

- **Concatenation**, + operator, creates a **new** list, with copies
- **Mutate** list with `L.extend(some_list)` (copy of `some_list`)

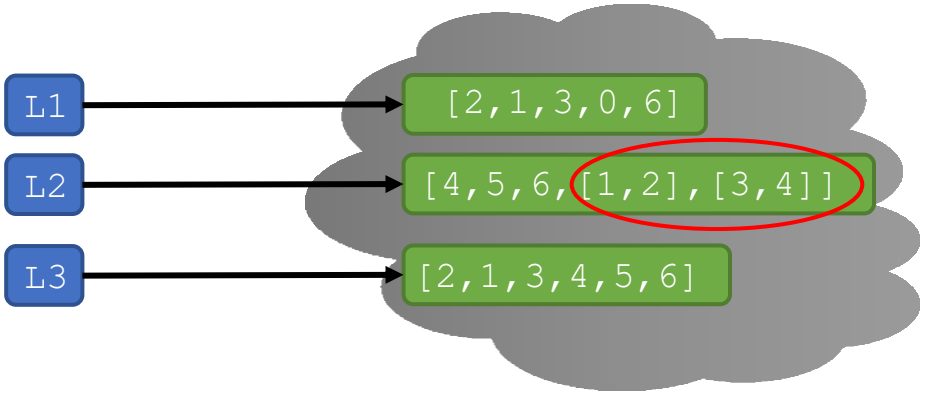
```
L1 = [2,1,3]
L2 = [4,5,6]
L3 = L1 + L2                    → L3 is [2,1,3,4,5,6]
L1.extend([0,6])                → mutate L1 to [2,1,3,0,6]
L2.extend([[1,2],[3,4]])        → mutate L2 to[4,5,6,[1,2],[3,4]]
```



COMBINING LISTS

- **Concatenation**, + operator, creates a **new** list, with copies
- **Mutate** list with `L.extend(some_list)` (copy of `some_list`)

```
L1 = [2,1,3]
L2 = [4,5,6]
L3 = L1 + L2                    → L3 is [2,1,3,4,5,6]
L1.extend([0,6])                → mutate L1 to [2,1,3,0,6]
L2.extend([[1,2],[3,4]])        → mutate L2 to [4,5,6,[1,2],[3,4]]
```



Extending by a list of lists gives us new list elements

TRICKY EXAMPLE 3: combining

```
L = [1,2,3,4]
for e in L:
    L = L + L
    print(L)
```

Originally [1,2,3,4]

L is **bound to a new object** each iteration; but looping of e walks down structure pointed to when called, so iterates only 4 times, over original [1,2,3,4]

1st time: **new** L is [1, 2, 3, 4, 1, 2, 3, 4]

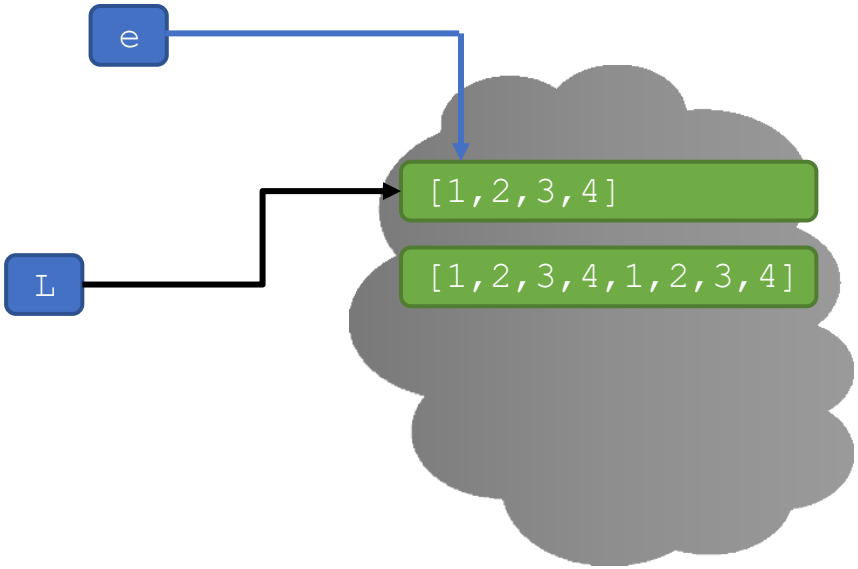
2nd time: **new** L is [1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4]

3rd time: **new** L is [1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4
1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4]

4th time: **new** L is [1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4
1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4
1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4
1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4]

TRICKY EXAMPLE 3: combining

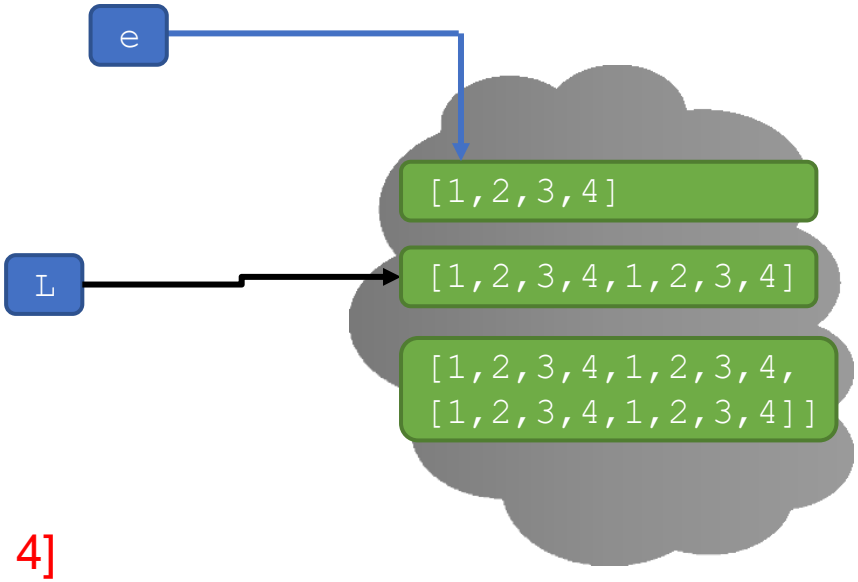
```
L = [1,2,3,4]
for e in L:
  L = L + L
  print(L)
```



TRICKY EXAMPLE 3: combining

Note: e is still indexing into original data structure

```
L = [1,2,3,4]
for e in L:
    L = L + L
    print(L)
```

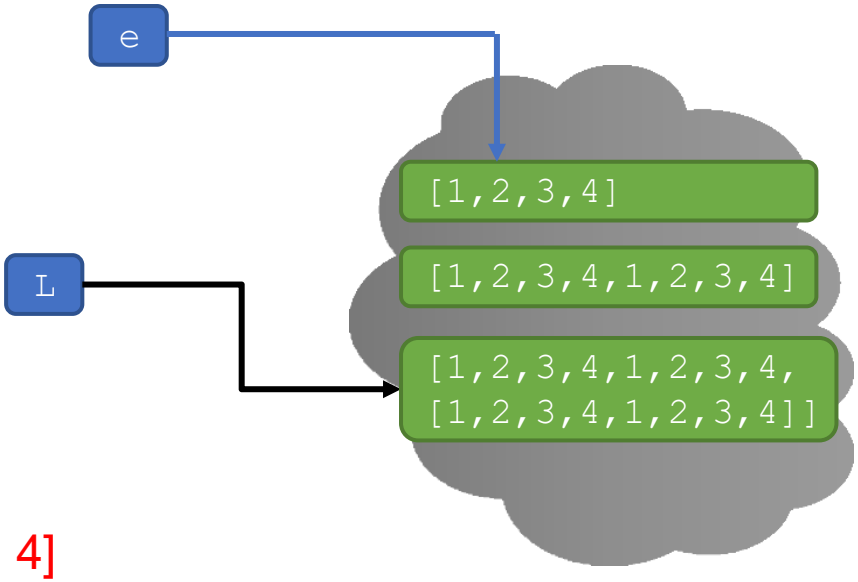


1st time: **new** L is [1, 2, 3, 4, 1, 2, 3, 4]

TRICKY EXAMPLE 3: combining

Note: e is still indexing into original data structure

```
L = [1,2,3,4]
for e in L:
    L = L + L
    print(L)
```



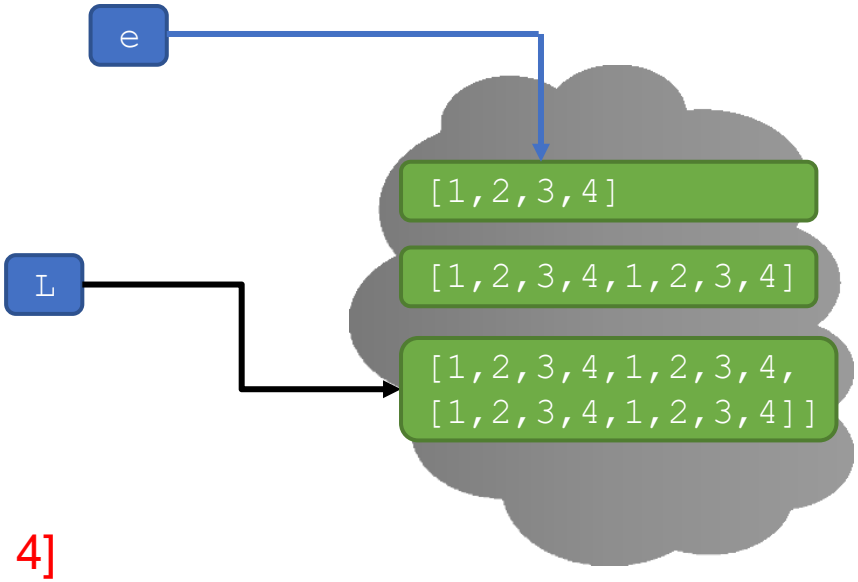
1st time: **new** L is [1, 2, 3, 4, 1, 2, 3, 4]

2nd time: **new** L is [1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4]

TRICKY EXAMPLE 3: combining

Note: e is still indexing into original data structure

```
L = [1,2,3,4]
for e in L:
  L = L + L
  print(L)
```



1st time: **new** L is [1, 2, 3, 4, 1, 2, 3, 4]

2nd time: **new** L is [1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4]

TRICKY EXAMPLE 3: combining

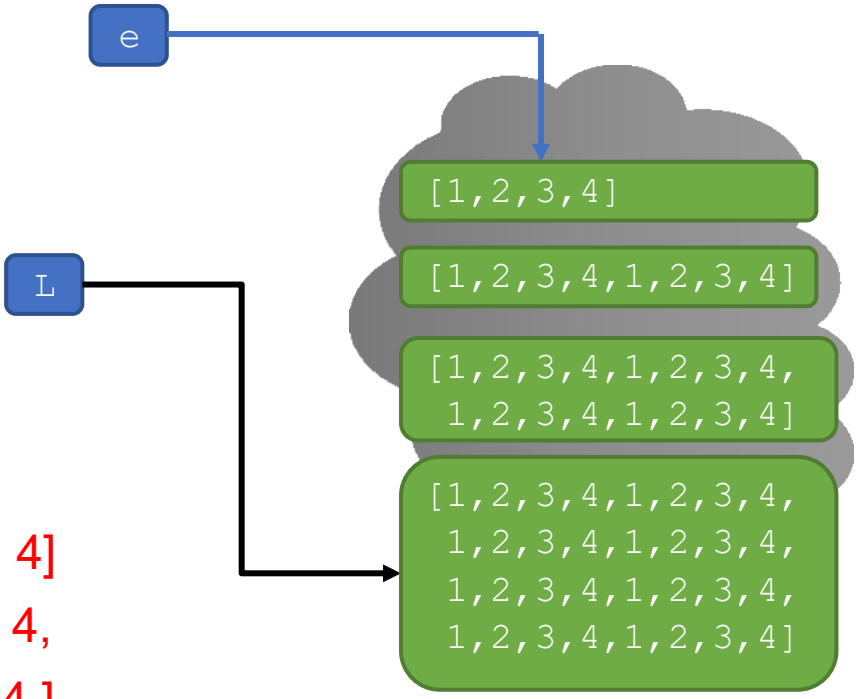
Note: e is still indexing into original data structure

```
L = [1,2,3,4]
for e in L:
    L = L + L
    print(L)
```

1st time: **new** L is [1, 2, 3, 4, 1, 2, 3, 4]

2nd time: **new** L is [1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4]

3rd time: **new** L is [1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4 ,
1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4]



TRICKY EXAMPLE 3: combining

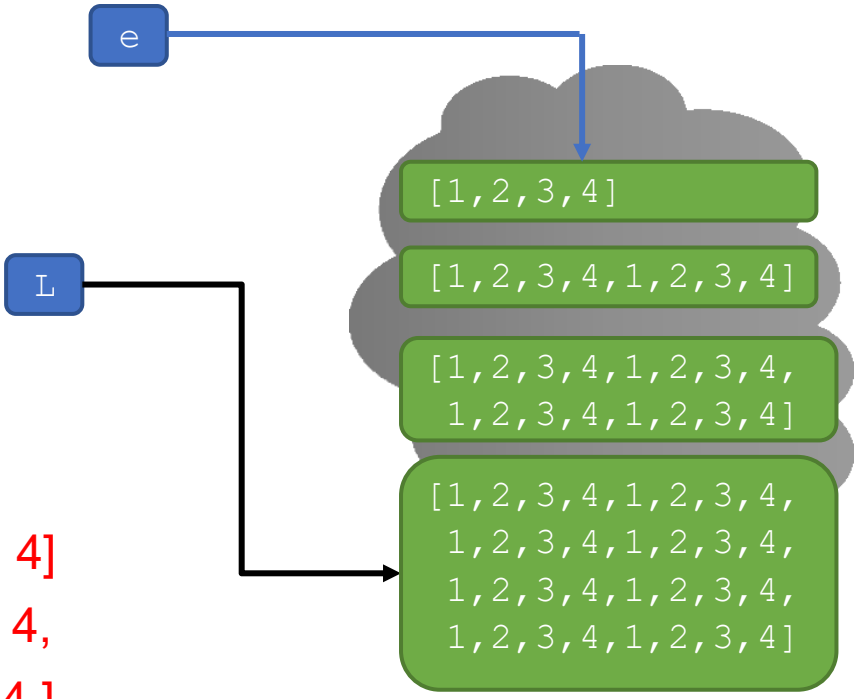
Note: e is still indexing into original data structure

```
L = [1,2,3,4]
for e in L:
  L = L + L
  print(L)
```

1st time: **new** L is [1, 2, 3, 4, 1, 2, 3, 4]

2nd time: **new** L is [1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4]

3rd time: **new** L is [1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4 ,
1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4]

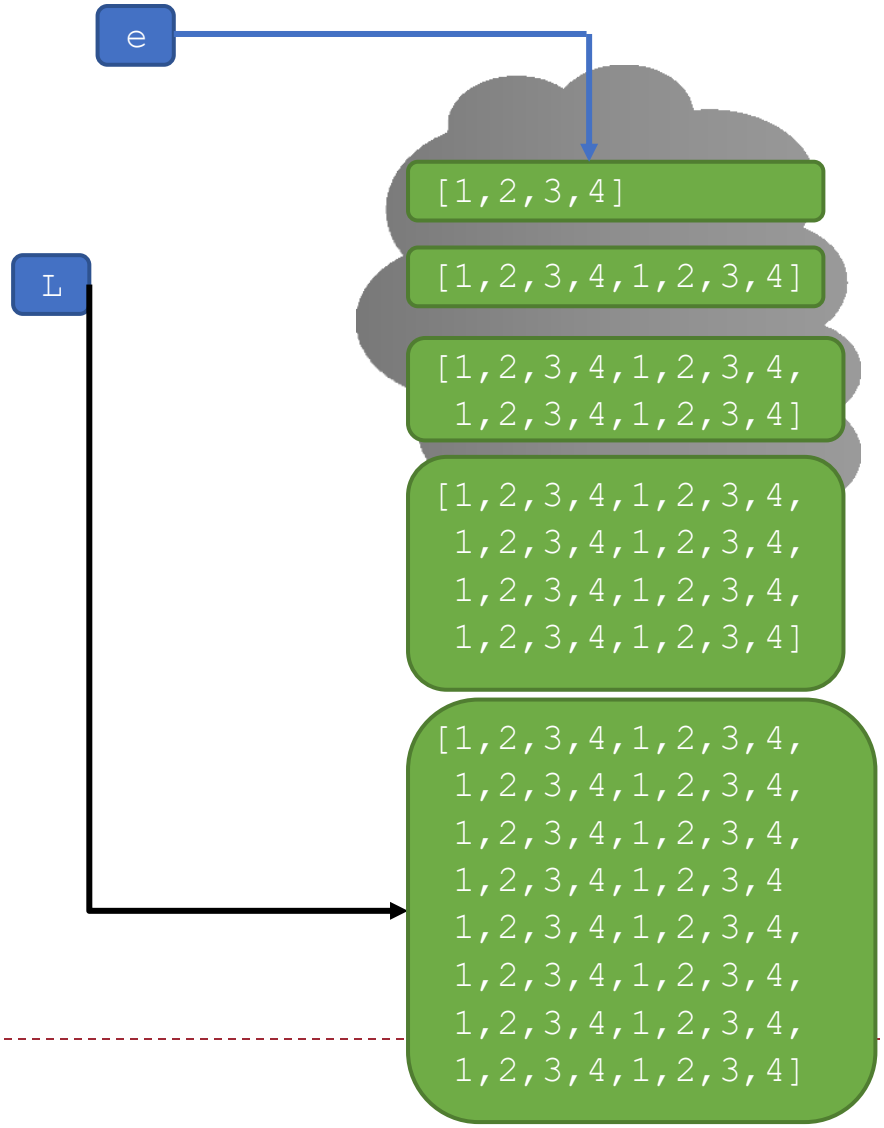


TRICKY EXAMPLE 3: combining

Note: e is still indexing into original data structure

```
L = [1,2,3,4]
for e in L:
    L = L + L
    print(L)
```

4th time: **new** L is [1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4,
1, 2, 3, 4, 1, 2, 3, 4]



TRICKY EXAMPLE 3: combining

```
L = [1,2,3,4]
for e in L:
  L = L + L
  print(L)
```

```
L = [1,2,3,4]
for e in L:
  L.extend(L)
  print(L)
```

EMPTY OUT A LIST AND CHECKING THAT IT'S THE SAME OBJECT

- You can **mutate a list to remove all its elements**
  - This **does not make a new empty list**!
- Use `L.clear()`
- How to check that it's **the same object in memory**?
  - Use the id() function
  - Try this in the console

Same!

```
>>> L = [4,5,6]
>>> id(L)
>>> L.append(8)
>>> id(L)
>>> L.clear()
>>> id(L)
```

```
>>> L = [4,5,6]
>>> id(L)
>>> L.append(8)
>>> id(L)
>>> L = []
>>> id(L)
```

Different!

ALIASING, CLONING

MAKING A COPY OF THE LIST

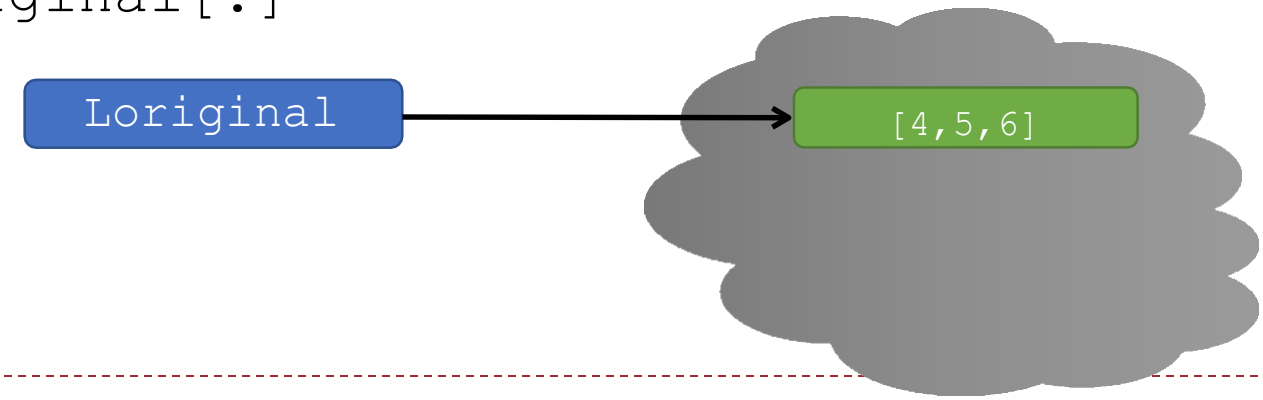
- Can **make a copy of a list object** by duplicating all elements (top-level) into a new list object
- `Lcopy = L[:]`
  - Equivalent to looping over L and appending each element to Lcopy
  - This does not make a copy of elements that are lists (will see how to do this at the end of this lecture)

```
Loriginal = [4,5,6]
Lnew = Loriginal[:]
```

MAKING A COPY OF THE LIST

- Can **make a copy of a list object** by duplicating all elements (top-level) into a new list object
- `Lcopy = L[:]`
  - Equivalent to looping over L and appending each element to Lcopy
  - This does not make a copy of elements that are lists (will see how to do this at the end of this lecture)

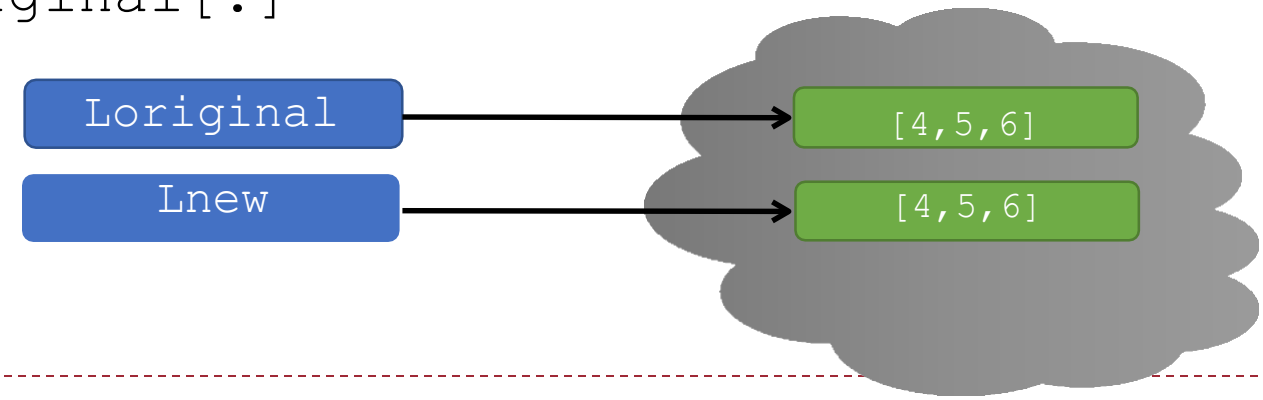
```
Loriginal = [4,5,6]
Lnew = Loriginal[:]
```



MAKING A COPY OF THE LIST

- Can **make a copy of a list object** by duplicating all elements (top-level) into a new list object
- `Lcopy = L[:]`
  - Equivalent to looping over L and appending each element to Lcopy
  - This does not make a copy of elements that are lists (will see how to do this at the end of this lecture)

```
Loriginal = [4,5,6]
Lnew = Loriginal[:]
```



YOU TRY IT!

- Write a function that meets the specification.
- Hint. Make a copy to save the elements. Then use L.clear() to empty out the list and repopulate it with the ones you're keeping.

```
def remove_all(L, e):
    """
    L is a list
    Mutates L to remove all elements in L that are equal to e
    Returns None
    """

L = [1,2,2,2]
remove_all(L, 2)
print(L) # prints [1]
```

OPERATION ON LISTS: `remove`

- Delete element at a **specific index** with `del(L[index])`
- Remove element at **end of list** with `L.pop()`, returns the removed element (can also call with specific index: `L.pop(3)`)
- Remove a **specific element** with `L.remove(element)`
  - Looks for the element and removes it (mutating the list)
  - If element occurs multiple times, removes first occurrence
  - If element not in list, gives an error

```
L = [2,1,3,6,3,7,0] # do below in order
L.remove(2)     → mutates L = [1,3,6,3,7,0]
L.remove(3)     → mutates L = [1,6,3,7,0]
del(L[1])       → mutates L = [1,3,7,0]
a = L.pop()     → returns 0 and mutates L = [1,3,7]
```

OPERATION ON LISTS: `remove`

- Delete element at a **specific index** with `del(L[index])`
- Remove element at **end of list** with `L.pop()`, returns the removed element (can also call with specific index: `L.pop(3)`)
- Remove a **specific element** with `L.remove(element)`
  - Looks for the element and removes it (mutating the list)
  - If element occurs multiple times, removes first occurrence
  - If element not in list, gives an error

```
L = [2,1,3,6,3,7,0] # do below in order
L.remove(2)     → mutates L = [1,3,6,3,7,0]
L.remove(3)     → mutates L = [1,6,3,7,0]
del(L[1])       → mutates L = [1,3,7,0]
a = L.pop()     → returns 0 and mutates L = [1,3,7]
```

OPERATION ON LISTS: `remove`

- Delete element at a **specific index** with `del(L[index])`
- Remove element at **end of list** with `L.pop()`, returns the removed element (can also call with specific index: `L.pop(3)`)
- Remove a **specific element** with `L.remove(element)`
  - Looks for the element and removes it (mutating the list)
  - If element occurs multiple times, removes first occurrence
  - If element not in list, gives an error

```
L = [2,1,3,6,3,7,0] # do below in order
L.remove(2)     → mutates L = [1,3,6,3,7,0]
L.remove(3)     → mutates L = [1,6,3,7,0]
del(L[1])       → mutates L = [1,3,7,0]
a = L.pop()     → returns 0 and mutates L = [1,3,7]
```

OPERATION ON LISTS: `remove`

- Delete element at a **specific index** with `del(L[index])`
- Remove element at **end of list** with `L.pop()`, returns the removed element (can also call with specific index: `L.pop(3)`)
- Remove a **specific element** with `L.remove(element)`
  - Looks for the element and removes it (mutating the list)
  - If element occurs multiple times, removes first occurrence
  - If element not in list, gives an error

```
L = [2,1,3,6,3,7,0] # do below in order
L.remove(2)     → mutates L = [1,3,6,3,7,0]
L.remove(3)     → mutates L = [1,6,3,7,0]
del(L[1])       → mutates L = [1,3,7,0]
a = L.pop()     → returns 0 and mutates L = [1,3,7]
```

OPERATION ON LISTS: `remove`

- Delete element at a **specific index** with `del(L[index])`
- Remove element at **end of list** with `L.pop()`, returns the removed element (can also call with specific index: `L.pop(3)`)
- Remove a **specific element** with `L.remove(element)`
  - Looks for the element and removes it (mutating the list)
  - If element occurs multiple times, removes first occurrence
  - If element not in list, gives an error

all these operations **mutate** the list

```
L = [2,1,3,6,3,7,0] # do below in order
L.remove(2)     → mutates L = [1,3,6,3,7,0]
L.remove(3)     → mutates L = [1,6,3,7,0]
del(L[1])       → mutates L = [1,3,7,0]
a = L.pop()     → returns 0 and mutates L = [1,3,7]
```

EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR

- Rewrite the code to remove `e` as long as we still had it in the list
- It works well!

```
def remove_all(L, e):
  """
  L is a list
  Mutates L to remove all elements in L that are equal to e
  Returns None.
  """
  while e in L:
      L.remove(e)
```

EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR

- What if the code was this:

```
def remove_all(L, e): """
    L is a list
    Mutates L to remove all elements in L that are equal to e
    Returns None.
    """
    for elem in L:
        if elem == e:
            L.remove(e)

L = [1,2,2,2]
remove_all(L, 2)
print(L)     # should print [1]
```

EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR

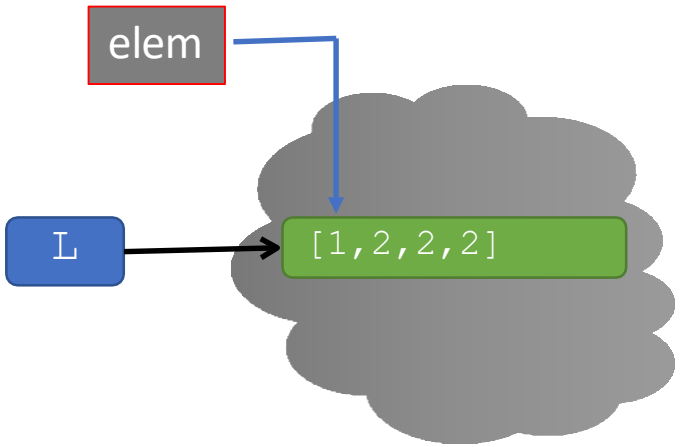
- What if the code was this:

```
def remove_all(L, e): """
    L is a list
    Mutates L to remove all elements in L that are equal to e
    Returns None.
    """
    for elem in L:
        if elem == e:
            L.remove(e)

L = [1,2,2,2]
remove_all(L, 2)
print(L)      # should print [1]
```

Actually prints [1,2]

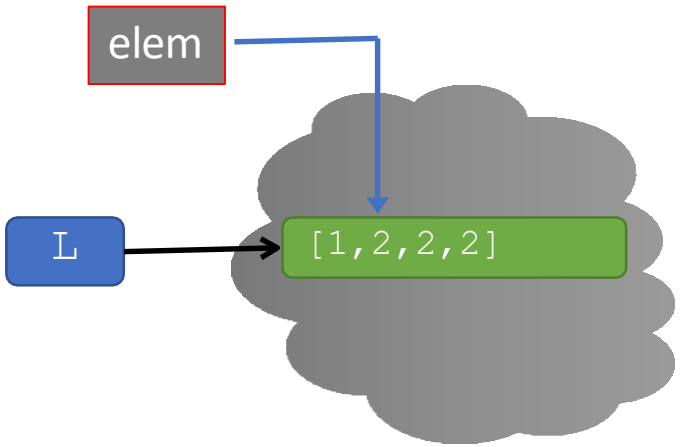
EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR

```
def remove_all(L, e): """
    L is a list
    Mutates L to remove all elements in L that are equal to e
    Returns None.
    """
    for elem in L:
        if elem == e:
            L.remove(e)

L = [1,2,2,2]
remove_all(L, 2)
print(L)     # should print [1]
```



EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR

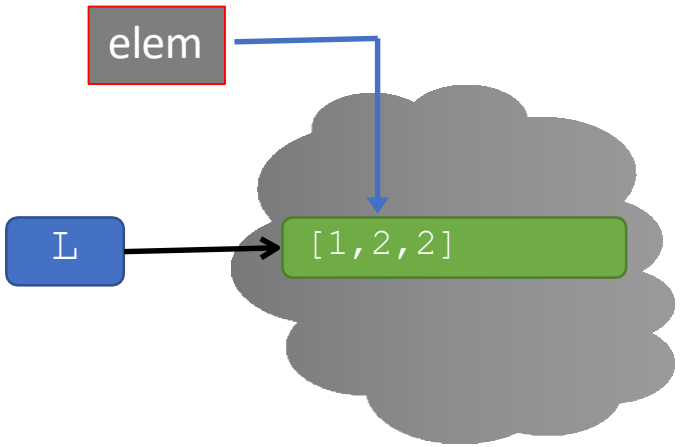
```
def remove_all(L, e): """
    L is a list
    Mutates L to remove all elements in L that are equal to e
    Returns None.
    """
    for elem in L:
        if elem == e:
            L.remove(e)

L = [1,2,2,2]
remove_all(L, 2)
print(L)     # should print [1]
```



EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR

```
def remove_all(L, e): """
    L is a list
    Mutates L to remove all elements in L that are equal to e
    Returns None.
    """
    for elem in L:
        if elem == e:
            L.remove(e)
```

Removes the 2, but doesn't shift the pointer back!

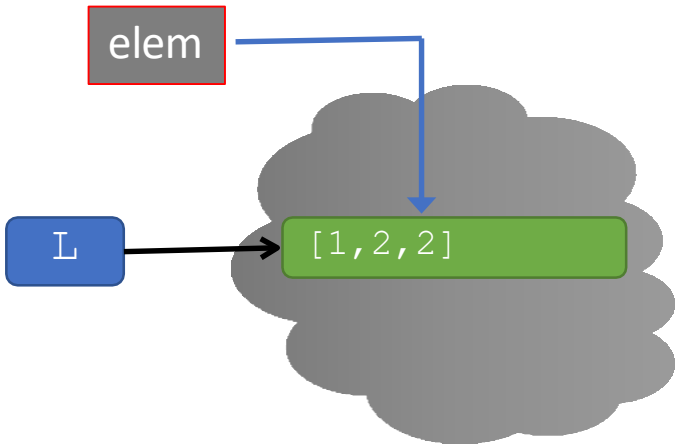



```
L = [1,2,2,2]
remove_all(L, 2)
print(L)     # should print [1]
```

EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR

```
def remove_all(L, e): """
    L is a list
    Mutates L to remove all elements in L that are equal to e
    Returns None.
    """
    for elem in L:
        if elem == e:
            L.remove(e)

L = [1,2,2,2]
remove_all(L, 2)
print(L)     # should print [1]
```

elem moves forward in the sequence



EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR

- It's not correct! We **removed items as we iterated over the list**!

```
def remove_all(L, e): """
    L is a list
    Mutates L to remove all elements in L that are equal to e
    Returns None.
    """
    for elem in L:
        if elem == e:
            L.remove(e)
```

Remove the 2, and done

```
L = [1,2,2,2]
remove_all(L, 2)
print(L)     # should print [1]
```